AI ASSISTED CODING

ASSIGNMENT-20.5

Name: Hari Priya

H.T.No:2303A51104

Task 1 – Password Storage Vulnerability Check

Task:

Analyze an AI-generated signup script for insecure password storage.

Instructions:

• Prompt AI to generate a Python user registration program.

• Check whether passwords are stored in plain text.

• Improve the script using hashing (bcrypt or hashlib).

• A secure registration script with hashed password storage.

```python
#Analyze an AI-generated signup script for insecure paasword storage.
#Generate a python user registration program.
# Check whether passwords are stored in plain text.
# improve the script using hashing (bcrypt or hashlib).
import hashlib
import os
import binascii

# Simulated user database storing only hashed passwords
user_db = {}
# Secure password hashing helper
def hash_password(password):
    salt = os.urandom(16)
    pwd_hash = hashlib.pbkdf2_hmac('sha256', password.encode('utf-8'), salt, 100_000)
    return binascii.hexlify(salt + pwd_hash).decode('ascii')
# Simulated user registration function
def register_user(username, password):
    # Never store passwords in plain text.
    hashed_password = hash_password(password)
    user_db[username] = hashed_password
    # Simulate storing the hashed password (in a real application, this would be saved to a database)
    print(f"User '{username}' registered with hashed password: {hashed_password}")
    return True
# Example usage
username = "user1"
password = "password123" # Insecure password
register_user(username, password)
# Example of a secure password
secure_password = "StrongPassword!@#"
register_user(username, secure_password)
```

```
PS C:\Users\HARI PRIYA\Desktop\AIAC> & "C:\Users\HARI PRIYA\AppData\Local\Microsoft\WindowsApps\python3.11.exe" "c:/Users/HARI PRIYA/Desktop/AIAC/Lab_assignment-20-5.py"
User 'user1' registered with hashed password: eb2194937460211f05b61dace7327beae0b45e14bc433358eb123553f2ce4f9c4225f0af9f4d818d19fd6ce2143b623e
User 'user1' registered with hashed password: b4219da4be52b01eb5b88f4853e29682a6611fd18a6a37c4dcb4c1e845aa1a9a9763557c4efa3fdab9812a8cbaf3ab01
```

Task 2 – Hardcoded Secrets Detection

Task:

Evaluate AI-generated code for hardcoded API keys or credentials.

Instructions:

• Ask AI to generate a script that connects to an external API.

• Identify if API keys/passwords are written directly in code.

• Refactor using environment variables or config files.

Expected Output:

• A secure script with secrets managed properly.

```
#- Hardcoded Secrets Detection
#code for hardcoded API keys or credentials.
#generate a script that connects to an external API.
import requests
# Simulated API connection function
def connect_to_api(api_key):
    if api_key == "12345":  # Insecure hardcoded API key
        print("Connected to API with hardcoded key (insecure).")
    else:
        print("Connected to API with provided key (secure).")
# Example usage
hardcoded_api_key = "12345"  # Insecure hardcoded API key
connect_to_api(hardcoded_api_key)
# Example of a secure API key (in practice, this should be stored securely, e.g in environment variables)
secure_api_key = "abcdef123456"  # Simulated secure API key
connect_to_api(secure_api_key)
```

```
import os
import configparser
CONFIG_PATH = "config.ini"
ENV_API_KEY = "API_KEY"
INSECURE_API_KEY = "12345"
# Simulated API connection function
def connect_to_api(api_key):
    if api_key == INSECURE_API_KEY:
        print("Connected to API with hardcoded key (insecure).")
    else:
        print("Connected to API with provided key (secure).")
def load_api_key_from_env():
    return os.environ.get(ENV_API_KEY)
def load_api_key_from_config(path=CONFIG_PATH):
    config = configparser.ConfigParser()
    if not os.path.exists(path):
        return None
    config.read(path)
    return config.get("secrets", "api_key", fallback=None)
def get_api_key():
    api_key = load_api_key_from_env()
    if api_key:
        print(f"Loaded API key from environment variable {ENV_API_KEY}.")
        return api_key
    api_key = load_api_key_from_config()
    if api_key:
        print(f"Loaded API key from config file: {CONFIG_PATH}")
        return api_key
    raise RuntimeError(
        f"No API key found. Set the {ENV_API_KEY} environment variable or add it to {CONFIG_PATH}."
    )
# Example usage
try:
    api_key = get_api_key()
    connect_to_api(api_key)
except RuntimeError as error:
    print(error)
```

Task 3 – File Upload Security Validation

Task:

Test an AI-generated file upload program for unsafe file handling.

Instructions:

• Prompt AI to generate a Flask file upload example.

• Check for vulnerabilities like uploading executable files. • Add validation for file type, size, and secure filename handling.

Expected Output:

• A secure file upload implementation.

```python
#generate a Flask file upload example.
#Check for vulnerabilities like uploading executable files.
#Add validation for file type, size, and secure filename handling.
from flask import Flask, request, redirect, url_for, render_template
from werkzeug.utils import secure_filename
import os
app = Flask(__name__)
# Set the upload folder and allowed extensions
UPLOAD_FOLDER = 'uploads'
ALLOWED_EXTENSIONS = {'txt', 'pdf', 'png', 'jpg', 'jpeg', 'gif'}
app.config['UPLOAD_FOLDER'] = UPLOAD_FOLDER
# Function to check allowed file extensions
def allowed_file(filename):
    return '.' in filename and filename.rsplit('.', 1)[1].lower() in ALLOWED_EXTENSIONS
@app.route('/', methods=['GET', 'POST'])
def upload_file():
    if request.method == 'POST':
        # Check if the post request has the file part
        if 'file' not in request.files:
            return redirect(request.url)
        file = request.files['file']
        # If the user does not select a file, the browser submits an empty file without a filename
        if file.filename == '':
            return redirect(request.url)
        if file and allowed_file(file.filename):
            filename = secure_filename(file.filename)
            file.save(os.path.join(app.config['UPLOAD_FOLDER'], filename))
            return 'File uploaded successfully'
    return render_template('upload.html')
if __name__ == '__main__':
    if not os.path.exists(UPLOAD_FOLDER):
        os.makedirs(UPLOAD_FOLDER)
    app.run(debug=True)
```

Task 4 – Command Injection Prevention

Task:

Analyze an AI-generated Python script that executes system commands.

Instructions:

• Ask AI to generate code using os.system() or subprocess.

• Identify command injection risks with user input.

• Fix using safe subprocess calls and input validation.

Expected Output:

• A secure command execution script.

```python
#generate code using os.system() or subprocess.
#Identify command injection risks with user input.
#Fix using safe subprocess calls and input validation.
#Generate A secure command execution script
import subprocess
import shlex
def execute_command(user_input):
    # Validate user input to prevent command injection
    if not user_input.isalnum():
        raise ValueError("Invalid input: Only alphanumeric characters are allowed.")

    # Use shlex to safely split the command and arguments
    command = f"echo {user_input}"
    args = shlex.split(command)

    # Execute the command securely using subprocess
    result = subprocess.run(args, capture_output=True, text=True)

    return result.stdout
if __name__ == "__main__":
    user_input = input("Enter a string to echo: ")
    try:
        output = execute_command(user_input)
        print(f"Command output: {output}")
    except ValueError as e:
        print(e)
```

Task 5 – Secure Input Handling in Web Forms

Task:

Check AI-generated web form code for improper input validation.

Instructions:

• Ask AI to generate an HTML + Flask feedback form.

• Identify missing validation and unsafe rendering.

• Fix using server-side validation and sanitization.

Expected Output:

• A secure form resistant to injection attacks.

```python
#Secure Input Handling in Web Forms
#to generate an HTML + Flask feedback form.
#Identify missing validation and unsafe rendering.
#Fix using server-side validation and sanitization.
#A secure form resistant to injection attacks.
from flask import Flask, request, render_template_string
import re
app = Flask(__name__)
# HTML template for the feedback form
form_template = '''
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Feedback Form</title>
</head>
<body>
    <h1>Feedback Form</h1>
    <form method="POST" action="/submit">
        <label for="name">Name:</label>
        <input type="text" id="name" name="name" required><br><br>

        <label for="email">Email:</label>
        <input type="email" id="email" name="email" required><br><br>

        <label for="message">Message:</label>
        <textarea id="message" name="message" rows="5" cols="40" required></textarea><br><br>

        <input type="submit" value="Submit">
    </form>
</body>
</html>
'''
```

```python
# Route to display the feedback form
@app.route('/')
def index():
    return render_template_string(form_template)
# Route to handle form submission
@app.route('/submit', methods=['POST'])
def submit():
    name = request.form['name']
    email = request.form['email']
    message = request.form['message']
    # Server-side validation
    if not re.match(r'^[A-Za-z\s]+$', name):
        return "Invalid name. Only letters and spaces are allowed."
    if not re.match(r'^[\w\.-]+@[\w\.-]+\.\w+$', email):
        return "Invalid email address."
    if len(message) > 500:
        return "Message is too long. Maximum 500 characters allowed."
    # Sanitize input to prevent XSS
    name = re.sub(r'[<>]', '', name)
    email = re.sub(r'[<>]', '', email)
    message = re.sub(r'[<>]', '', message)
    return f"Thank you for your feedback, {name}! Your message has been received."
if __name__ == '__main__':
    app.run(debug=True)
```